

Lecture 20: Subset Sum

Read: Sections 34.5.5 in CLR.

Subset Sum: The Subset Sum problem (SS) is the following. Given a finite set S of positive integers $S = \{w_1, w_2, \dots, w_n\}$ and a *target value*, t , we want to know whether there exists a subset $S' \subseteq S$ that sums exactly to t .

This problem is a simplified version of the 0-1 Knapsack problem, presented as a decision problem. Recall that in the 0-1 Knapsack problem, we are given a collection of objects, each with an associated weight w_i and associated value v_i . We are given a knapsack of capacity W . The objective is to take as many objects as can fit in the knapsack's capacity so as to maximize the value. (In the fractional knapsack we could take a portion of an object. In the 0-1 Knapsack we either take an object entirely or leave it.) In the simplest version, suppose that the value is the same as the weight, $v_i = w_i$. (This would occur for example if all the objects were made of the same material, say, gold.) Then, the best we could hope to achieve would be to fill the knapsack entirely. By setting $t = W$, we see that the subset sum problem is equivalent to this simplified version of the 0-1 Knapsack problem. It follows that if we can show that this simpler version is NP-complete, then certainly the more general 0-1 Knapsack problem (stated as a decision problem) is also NP-complete.

Consider the following example.

$$S = \{3, 6, 9, 12, 15, 23, 32\} \quad \text{and} \quad t = 33.$$

The subset $S' = \{6, 12, 15\}$ sums to $t = 33$, so the answer in this case is yes. If $t = 34$ the answer would be no.

Dynamic Programming Solution: There is a dynamic programming algorithm which solves the Subset Sum problem in $O(n \cdot t)$ time.²

The quantity $n \cdot t$ is a polynomial function of n . This would seem to imply that the Subset Sum problem is in P. But there is an important catch. Recall that in all NP-complete problems we assume (1) running time is measured as a function of input size (number of bits) and (2) inputs must be encoded in a reasonable succinct manner. Let us assume that the numbers w_i and t are all b -bit numbers represented in base 2, using the fewest number of bits possible. Then the input size is $O(nb)$. The value of t may be as large as 2^b . So the resulting algorithm has a running time of $O(n2^b)$. This is polynomial in n , but exponential in b . Thus, this running time is not polynomial as a function of the input size.

Note that an important consequence of this observation is that the SS problem is not hard when the numbers involved are small. If the numbers involved are of a fixed number of bits (a constant independent of n), then the problem is solvable in polynomial time. However, we will show that in the general case, this problem is NP-complete.

SS is NP-complete: The proof that Subset Sum (SS) is NP-complete involves the usual two elements.

- (i) $SS \in NP$.
- (ii) Some known NP-complete problem is reducible to SS. In particular, we will show that Vertex Cover (VC) is reducible to SS, that is, $VC \leq_P SS$.

To show that SS is in NP, we need to give a verification procedure. Given S and t , the certificate is just the indices of the numbers that form the subset S' . We can add two b -bit numbers together in $O(b)$ time. So, in polynomial time we can compute the sum of elements in S' , and verify that this sum equals t .

For the remainder of the proof we show how to reduce vertex cover to subset sum. We want a polynomial time computable function f that maps an instance of the vertex cover (a graph G and integer k) to an instance of the subset sum problem (a set of integers S and target integer t) such that G has a vertex cover of size k if and only if S has a subset summing to t . Thus, if subset sum were solvable in polynomial time, so would vertex cover.

²We will leave this as an exercise, but the formulation is, for $0 \leq i \leq n$ and $0 \leq t' \leq t$, $S[i, t'] = 1$ if there is a subset of $\{w_1, w_2, \dots, w_i\}$ that sums to t' , and 0 otherwise. The i th row of this table can be computed in $O(t)$ time, given the contents of the $(i - 1)$ -st row.

How can we encode the notion of selecting a subset of vertices that cover all the edges to that of selecting a subset of numbers that sums to t ? In the vertex cover problem we are selecting vertices, and in the subset sum problem we are selecting numbers, so it seems logical that the reduction should map vertices into numbers. The constraint that these vertices should cover all the edges must be mapped to the constraint that the sum of the numbers should equal the target value.

An Initial Approach: Here is an idea, which does not work, but gives a sense of how to proceed. Let E denote the number of edges in the graph. First number the edges of the graph from 1 through E . Then represent each vertex v_i as an E -element bit vector, where the j -th bit from the left is set to 1 if and only if the edge e_j is incident to vertex v_i . (Another way to think of this is that these bit vectors form the rows of an *incidence matrix* for the graph.) An example is shown below, in which $k = 3$.

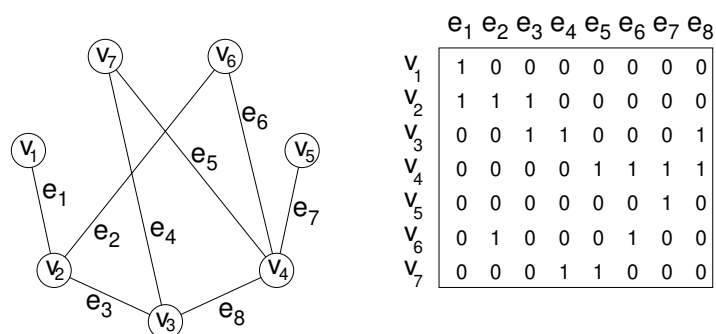


Fig. 54: Encoding a graph as a collection of bit vectors.

Now, suppose we take any subset of vertices and form the logical-or of the corresponding bit vectors. If the subset is a vertex cover, then every edge will be covered by at least one of these vertices, and so the logical-or will be a bit vector of all 1's, $1111 \dots 1$. Conversely, if the logical-or is a bit vector of 1's, then each edge has been covered by some vertex, implying that the vertices form a vertex cover. (Later we will consider how to encode the fact that there only allowed k vertices in the cover.)

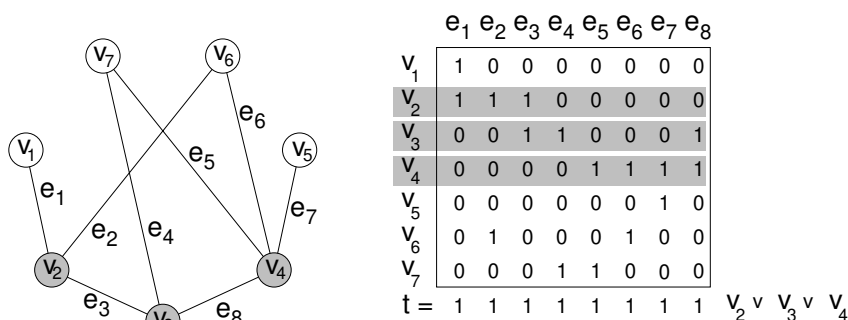


Fig. 55: The logical-or of a vertex cover equals $1111 \dots 1$.

Since bit vectors can be thought of as just a way of representing numbers in binary, this is starting to feel more like the subset sum problem. The target would be the number whose bit vector is all 1's. There are a number of problems, however. First, logical-or is not the same as addition. For example, if both of the endpoints of some edge are in the vertex cover, then its value in the corresponding column would be 2, not 1. Second, we have no way of controlling how many vertices go into the vertex cover. (We could just take the logical-or of all the vertices, and then the logical-or would certainly be a bit vectors of 1's.)

There are two ways in which addition differs significantly from logical-or. The first is the issue of carries. For example, the $1101 \vee 0011 = 1111$, but in binary $1101 + 0011 = 1000$. To fix this, we recognize that we do not have to use a binary (base-2) representation. In fact, we can assume any base system we want. Observe that each column of the incidence matrix has at most two 1's in any column, because each edge is incident to at most two vertices. Thus, if we use any base that is at least as large as base 3, we will never generate a carry to the next position. In fact we will use base 4 (for reasons to be seen below). Note that the base of the number system is just for our own convenience of notation. Once the numbers have been formed, they will be converted into whatever form our machine assumes for its input representation, e.g. decimal or binary.

The second difference between logical-or and addition is that an edge may generally be covered either once or twice in the vertex cover. So, the final sum of these numbers will be a number consisting of 1 and 2 digits, e.g. $1211 \dots 112$. This does not provide us with a unique target value t . We know that no digit of our sum can be a zero. To fix this problem, we will create a set of E additional *slack values*. For $1 \leq i \leq E$, the i th slack value will consist of all 0's, except for a single 1-digit in the i th position, e.g., 00000100000 . Our target will be the number $2222 \dots 222$ (all 2's). To see why this works, observe that from the numbers of our vertex cover, we will get a sum consisting of 1's and 2's. For each position where there is a 1, we can supplement this value by adding in the corresponding slack value. Thus we can boost any value consisting of 1's and 2's to all 2's. On the other hand, note that if there are any 0 values in the final sum, we will not have enough slack values to convert this into a 2.

There is one last issue. We are only allowed to place only k vertices in the vertex cover. We will handle this by adding an additional column. For each number arising from a vertex, we will put a 1 in this additional column. For each slack variable we will put a 0. In the target, we will require that this column sum to the value k , the size of the vertex cover. Thus, to form the desired sum, we must select exactly k of the vertex values. Note that since we only have a base-4 representation, there might be carries out of this last column (if $k \geq 4$). But since this is the last column, it will not affect any of the other aspects of the construction.

The Final Reduction: Here is the final reduction, given the graph $G = (V, E)$ and integer k for the vertex cover problem.

- (1) Create a set of n vertex values, $x_1, x_2, \dots, x_n$ using base-4 notation. The value x_i is equal to a 1 followed by a sequence of E base-4 digits. The j -th digit is a 1 if edge e_j is incident to vertex v_i and 0 otherwise.
- (2) Create E slack values $y_1, y_2, \dots, y_E$, where y_i is a 0 followed by E base-4 digits. The i -th digit of y_i is 1 and all others are 0.
- (3) Let t be the base-4 number whose first digit is k (this may actually span multiple base-4 digits), and whose remaining E digits are all 2.
- (4) Convert the x_i 's, the y_j 's, and t into whatever base notation is used for the subset sum problem (e.g. base 10). Output the set $S = \{x_1, \dots, x_n, y_1, \dots, y_E\}$ and t .

Observe that this can be done in polynomial time, in $O(E^2)$, in fact. The construction is illustrated in Fig. 56.

Correctness: We claim that G has a vertex cover of size k if and only if S has a subset that sums to t . If G has a vertex cover V' of size k , then we take the vertex values x_i corresponding to the vertices of V' , and for each edge that is covered only once in V' , we take the corresponding slack variable. It follows from the comments made earlier that the lower-order E digits of the resulting sum will be of the form $222 \dots 2$ and because there are k elements in V' , the leftmost digit of the sum will be k . Thus, the resulting subset sums to t .

Conversely, if S has a subset S' that sums to t then we assert that it must select exactly k values from among the vertex values, since the first digit must sum to k . We claim that these vertices V' form a vertex cover. In particular, no edge can be left uncovered by V' , since (because there are no carries) the corresponding column would be 0 in the sum of vertex values. Thus, no matter what slack values we add, the resulting digit position could not be equal to 2, and so this cannot be a solution to the subset sum problem.

		e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	
x_1	1	1	0	0	0	0	0	0	0	Vertex values
x_2	1	1	1	1	0	0	0	0	0	
x_3	1	0	0	1	1	0	0	0	1	
x_4	1	0	0	0	0	1	1	1	1	
x_5	1	0	0	0	0	0	0	1	0	
x_6	1	0	1	0	0	0	1	0	0	
x_7	1	0	0	0	1	1	0	0	0	
y_1	0	1	0	0	0	0	0	0	0	Slack values
y_2	0	0	1	0	0	0	0	0	0	
y_3	0	0	0	1	0	0	0	0	0	
y_4	0	0	0	0	1	0	0	0	0	
y_5	0	0	0	0	0	1	0	0	0	
y_6	0	0	0	0	0	0	1	0	0	
y_7	0	0	0	0	0	0	0	1	0	
y_8	0	0	0	0	0	0	0	0	1	
t		3	2	2	2	2	2	2	2	

vertex cover size ($k=3$)

Fig. 56: Vertex cover to subset sum reduction.

		e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	
x_1	1	1	0	0	0	0	0	0	0	Vertex values (take those in vertex cover)
x_2	1	1	1	1	0	0	0	0	0	
x_3	1	0	0	1	1	0	0	0	1	
x_4	1	0	0	0	0	1	1	1	1	
x_5	1	0	0	0	0	0	0	1	0	
x_6	1	0	1	0	0	0	1	0	0	
x_7	1	0	0	0	1	1	0	0	0	
y_1	0	1	0	0	0	0	0	0	0	Slack values (take one for each edge that has only one endpoint in the cover)
y_2	0	0	1	0	0	0	0	0	0	
y_3	0	0	0	1	0	0	0	0	0	
y_4	0	0	0	0	1	0	0	0	0	
y_5	0	0	0	0	0	1	0	0	0	
y_6	0	0	0	0	0	0	1	0	0	
y_7	0	0	0	0	0	0	0	1	0	
y_8	0	0	0	0	0	0	0	0	1	
t		3	2	2	2	2	2	2	2	

vertex cover size

Fig. 57: Correctness of the reduction.

It is worth noting again that in this reduction, we needed to have large numbers. For example, the target value t is at least as large as $4^E \geq 4^n$ (where n is the number of vertices in G). In our dynamic programming solution $W = t$, so the DP algorithm would run in $\Omega(n4^n)$ time, which is not polynomial time.

Lecture 21: Approximation Algorithms: VC and TSP

Read: Chapt 35 (up through 35.2) in CLRS.

Coping with NP-completeness: With NP-completeness we have seen that there are many important optimization problems that are likely to be quite hard to solve exactly. Since these are important problems, we cannot simply give up at this point, since people do need solutions to these problems. How do we cope with NP-completeness:

Use brute-force search: Even on the fastest parallel computers this approach is viable only for the smallest instances of these problems.

Heuristics: A *heuristic* is a strategy for producing a valid solution, but there are no guarantees how close it is to optimal. This is worthwhile if all else fails, or if lack of optimality is not really an issue.

General Search Methods: There are a number of very powerful techniques for solving general combinatorial optimization problems that have been developed in the areas of AI and operations research. These go under names such as *branch-and-bound*, *A*-search*, *simulated annealing*, and *genetic algorithms*. The performance of these approaches varies considerably from one problem to problem and instance to instance. But in some cases they can perform quite well.

Approximation Algorithms: This is an algorithm that runs in polynomial time (ideally), and produces a solution that is within a guaranteed factor of the optimum solution.

Performance Bounds: Most NP-complete problems have been stated as decision problems for theoretical reasons. However underlying most of these problems is a natural optimization problem. For example, the TSP optimization problem is to find the simple cycle of minimum cost in a digraph, the VC optimization problem is to find the vertex cover of minimum size, the clique optimization problem is to find the clique of maximum size. Note that sometimes we are minimizing and sometimes we are maximizing. An approximation algorithm is one that returns a legitimate answer, but not necessarily one of the smallest size.

How do we measure how good an approximation algorithm is? We define the *ratio bound* of an approximation algorithm as follows. Given an instance I of our problem, let $C(I)$ be the cost of the solution produced by our approximation algorithm, and let $C^*(I)$ be the optimal solution. We will assume that costs are strictly positive values. For a minimization problem we want $C(I)/C^*(I)$ to be small, and for a maximization problem we want $C^*(I)/C(I)$ to be small. For any input size n , we say that the approximation algorithm achieves *ratio bound* $\rho(n)$, if for all I , $|I| = n$ we have

$$\max_I \left(\frac{C(I)}{C^*(I)}, \frac{C^*(I)}{C(I)} \right) \leq \rho(n).$$

Observe that $\rho(n)$ is always greater than or equal to 1, and it is equal to 1 if and only if the approximate solution is the true optimum solution.

Some NP-complete problems can be approximated arbitrarily closely. Such an algorithm is given both the input, and a real value $\epsilon > 0$, and returns an answer whose ratio bound is at most $(1 + \epsilon)$. Such an algorithm is called a *polynomial time approximation scheme* (or *PTAS* for short). The running time is a function of both n and ϵ . As ϵ approaches 0, the running time increases beyond polynomial time. For example, the running time might be $O(n^{1/\epsilon})$. If the running time depends only on a polynomial function of $1/\epsilon$ then it is called a *fully polynomial-time approximation scheme*. For example, a running time like $O((1/\epsilon)^2 n^3)$ would be such an example, whereas $O(n^{1/\epsilon})$ and $O(2^{(1/\epsilon)} n)$ are not.

Although NP-complete problems are equivalent with respect to whether they can be solved exactly in polynomial time in the worst case, their approximability varies considerably.